CPSC 314 Assignment 2: Transformations

Due Oct 13, 2026

1 Introduction

In this Assignment you will utilize your knowledge of transformations to make things move. Here we will study how to build and animate with object hierarchies.

1.1 Template

- `A2.html` is the launcher of the assignment. Open it in your preferred browser to run the assignment, to get started.
- `A2.js` contains the JavaScript code used to set up the scene and the rendering environment. You will do most of your work here for this assignment.
- `glsl/` contains the vertex and fragment shaders for the eye.
- `js/` contains the required JavaScript libraries. You do not need to change anything here.
- `glb/` contains the geometric model which needs to be loaded in the scene.
- `images/` contains the texture images used.

2 Work to be done (100 points)

Here we assume you already have a working development environment which allows you to run your code from a local server. (if you do not, check out instructions from Assignment 1 for details). Once you have set up the environment, Study the template to get a sense of how it works.

a. **(5 points)** Load the glTF armadillo model.

In this assignment, we'll shift from a OBJ model to a glTF model which is more powerful. A rigged glTF armadillo is provided in `glb/`. Your task is to load the armadillo into the scene as in Figure 1. Specifically, fill in function `loadGLTFAsync()` in `js/setup.js` and make necessary changes to `A2.js`. You can check THREE.js's website for glTF loader



Figure 1: Question 1.a. Starter scene.

and function `loadOBJAsync()` for reference.
Resource: https://threejs.org/docs/examples/en/loaders/GLTFLoader

b. **(15 points)** Angry Boxing Gloves.

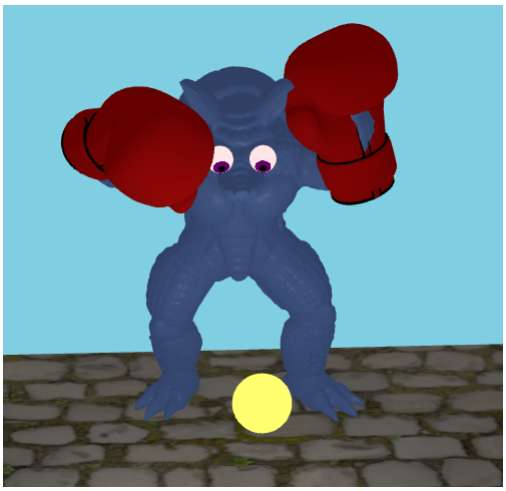


Figure 2: Question 1.b: Give the Armadillo gloves.

For this task, after placing the Armadillo on the scene, add some boxing gloves to scene on the Armadillo's hands. Unlike in A1, we can leverage the capability of glTF models to actually attach the boxing gloves to the Armadillo's hands! To prove that the gloves are attached, animate the Armadillo hand to wave around, increasing in speed when the sphere approaches closer! Refer to Figure 2 for what this may look like.

Hint 1: For attaching the boxing gloves, you'll most likely need to access the Armadillo's

skeletal structure. To do this, you can modify the `postLoading` function passed to `loadGLTFAsync()`.

Hint 2: Using `scene.add()` allows you to add an object as a child of the scene (attaching to the scene). If you access the armadillo's skeletal structure, you can add the boxing glove as a child to the armadillo's bone in the same way.

Hint 3: You may want to use `clock` variable to implement the Armadillo's periodic hand waving (maybe a `sin` function?)

c. **(10 points)** Armadillos have Eyes.



Figure 3: Question 1.c: Give the Armadillo eyes.

Your task here is to bless the Armadillo with a pair of googly eyes. One eyeball model is already provided. Your job is to place two copies of this eyeball onto where the eyeballs should be on the Armadillo with appropriate positions and scale (rotations are not necessary for now, see Hint 2). The result should look similar to Figure 3.

Hint 1: You can do this entire question in `A2.js`.
Hint 2: Don't worry if the eyes are initially pointing backwards into the Armadillo's head; in the next question you will have the opportunity to orient them correctly.

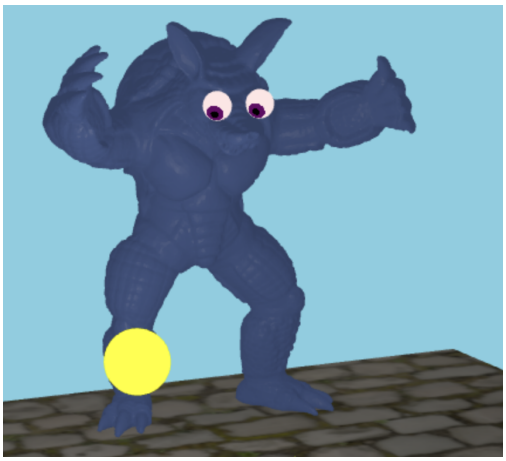


Figure 4: Question 1.d: Rotate the eyes to look at the sphere.

d. **(10 points)** Staring at the sphere. What good are eyes if they don't `lookAt` anything? Direct the Armadillo's gaze towards the nefarious floating sphere. Ensure that the eyes look at the sphere as it moves around the scene as shown in Figure 4. Also, notice that in Figure 4 each eye contains a pupil and an iris that help you with verifying your `lookAt` works.

Hint 1: `THREE.Matrix4 and THREE.Object3D` have a method called `lookAt` that may be of use.

e. **(20 points)** Laser eyes! Arm the Armadillo with eye lasers. Shoot the lasers from the Armadillo's eyes at the sphere when it gets too close. A distance threshold (units in world space) named `LaserDistance` is provided in `A2.js`. To be more specific, create two lasers from the eyes to the sphere. You need to create the geometries and choose an appropriate material by yourself. Figure 5 shows a possible result.

Hint 1: You may use any kind of material for the lasers (even a `ShaderMaterial`).
Hint 2: Note that the distance threshold between the sphere and the eye is given in the world frame, but the scale of the laser will likely be in a different frame.

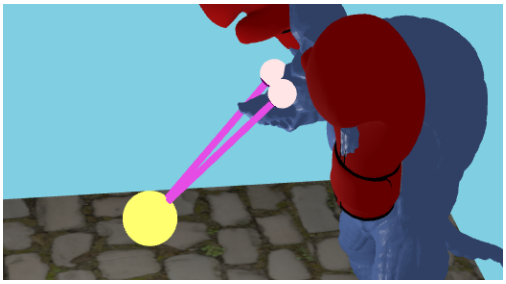


Figure 5: Question 1.e: Give the Armadillo laser eyes!

f. **(25 points)** Grow the ball! When the sphere is hit by the Armadillo's laser eyes, it should expand dramatically in size and shift colour slowly. Figure 6 shows how this should look.
Hint 1: It may be helpful to use `Color.lerp` to help interpolate between colors as the sphere grows.
Resource: `https://threejs.org/docs/#api/en/math/Color.lerp`

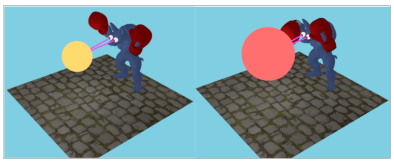


Figure 6: Question 1.f: Grow the Ball!

g. **15 pts** Feature Extension. In this part, you task is to extend the assignment to add features of your own choosing. The goal is to encourage you to explore the capabilities of Three.js and WebGL, and more importantly, unleash your creativity! The extension has to be non-trival to receive credit, but for full credit, the features have to be truly creative. See Sec. 4.2 for grading expectations. A small number of exceptional extensions will be selected from this group and shown in class, with the student's permission. You can check Assignment 1's spec document for ideas to get you started, and feel free to come up with your own ideas.

   First duplicate your current work into a new directory named 'part2', then implement your feature in this new directory. Write a brief description of your feature in the README file. You will be graded on both how it works and how well you can explain your feature.

3 Submission Instructions

3.1 Directory Structure

Your submission should contain two subdirectories - the first should be named 'part1' and should contain all parts except the feature extension; the second subdirectory should be named 'part2' and should contain your feature extenstion. Under the root directory of your assignment, please add both subdirectories including all the source files and everything else required to run each part in the respective folder. Do not create more sub-directories than the ones already provided.

You must also write a clear `README.txt` file which includes your name, student number, and CWL username, instructions on how to use the program (keyboard actions, etc.) and any information you would like to pass on to the marker. Place the file under the root directory of your assigment.

3.2 Submission Methods

Please compress everything under the root directory of your assignment into `a2.zip` and submit it on Canvas. You can make multiple submissions, but we will grade only the last one.

4 Grading

4.1 Face-to-face (F2F) Grading

Submitting the assignment is not the end of the game; to get a grade for the assignment, you must meet face-to-face with a TA in a 12-min slot during or outside lab hours, on Zoom, to demonstrate that you understand how your program works; Grading slots and detailed instructions on how to sign up and conduct F2F will be announced on Canvas and on Piazza. During the meeting, the TA will (1) ask you to run your code and inspect the correctness of the program; (2) ask you to explain parts of your code; (3) ask you some

questions about the assignment, and you will need to answer them in a limited timeframe. The questions will mostly be based on ThreeJS or WebGL concepts that you must have come across while working on the assignment; but also you may get conceptual questions based on lecture materials that are relevant to the assignment, or technicalities that you may not have thought about unless you really "digged deep" into the assignment. But no need to be nerveous! We evaluate your response based mainly if not entirely on the coherence of your thoughts, rather than how complete or long your response is: e.g. if the full answer to a question includes A+B+C+D and you mentioned A+B only in the provided time, but your thread of thought is logical, then you may still get full marks.

4.2 Point Allocation

All questions before Feature Extension have a total of 85 points. The points are warranted based on

- The functional correctness of your program, i.e., how visually close your results are to expected results;
- The algorithmic correctness of your program, e.g., applying transformation matrices in the right order;
- Your demonstrating your understanding of the code with your answers to TAs' questions during face-to-face (F2F) grading.

The feature extension grade is based on creativity and sophistication of your submission. We warrant partial marks on four scales:

- 15: Outstandingly creative and sophisticated feature extension. But note that only a very small number of extensions may get this score;
- 10: Average creativity and sophistication, e.g. adding a few interesting objects into the scene and animating, deforming the objects in visually appealing ways;
- 5: Basic creativity and sophistication, e.g. adding a few static objects into the scene, changing the orb's shape in simple ways;
- 0: Trivial or no feature extension, e.g. just changing the color of the orb.

4.3 Collaboration Policy

Submissions must be individual, but you are encouraged to discuss the assignment with your classmates, and you are allowed to use generative AI tools, e.g. ChatGPT, Claude to help with constructing your solution. **In the `README.txt` file, include names of the people you discussed the assignment with, websites you acquired information from, and if you used generative AI tools, include the names of the tools you used.** And you must understand your code and be able to explain them during F2F grading. If you cannot demonstrate at least a basic understanding of your code during face-to-face grading, you may

receive a zero for the assignment—even if your code runs perfectly. Hiring a tutor to write your assignment, or copy-pasting others' code without proper referencing in `README.txt` are considered as plagiarism, and will be reported to the dean's office.

4.4 Penalties

Aside from penalties from incorrect solution or plagiarism, we may apply the following penalties to each assignment:

Late penalty. You are entitled up to three grace (calendar) days in total throughout the term. No penalties would be applied for using them. However once you have used up the grace days, a deduction of 10 points would be applied to each extra late day. Note that

1. The three grace days are given for all assignments, **not per assignment**, so please use them wisely;
2. We check the time of your last submission to determine if you are late or not.

No-show penalty. Please sign up for a grading slot on the provided sign-up spreadsheet (link will be posted later on Piazza) before the submission deadline of the assignment, and show up to your slot on time. A 10-point deduction would be applied to each of the following circumstances:

1. Not signing up a grading slot before the sign-up period closes. Unless otherwise stated, the period closes at the same time as the submission deadline.
2. Not showing up at your grading slot.

If none of the provided slots work for you, or if you have already missed your slot, follow instructions outlined on piazza to get graded after the F2F grading period ends. Also, please note that

1. you'll need to explain to your TA why you're getting graded late, and may be asked to present documents to justify your hardship. The TA may remove the no-show penalty as long as he/she deems the justification to be reasonable.
2. In the past some students reported that their names disappeared mysteriously due to technical glitches, and the spreadsheet's edit history has no trace of it. So double check that your name is on the sign-up sheet after you sign up by refreshing the page.